

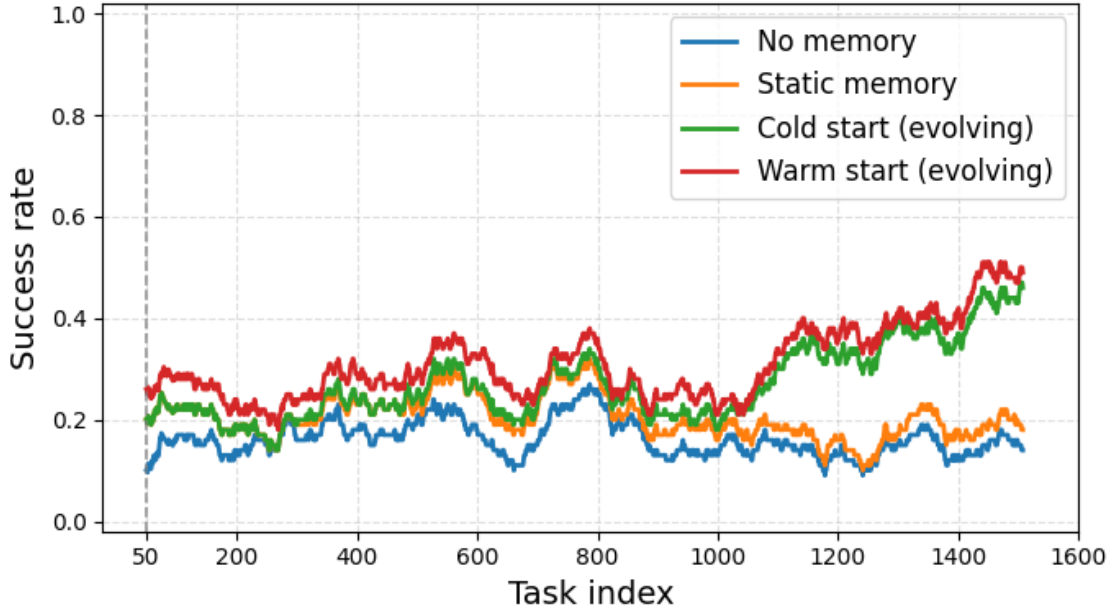


Figure 3 | **Memory-driven performance evolution on CyberGym.** Moving-average success rate (window size 100) tasks for four memory configurations using Gemini-2.5-Pro.

of exploitation iterations. Specifically, we take iterations from 1 to 20 and run on Cybench with two Gemini models, as shown in Figure 2. Both Gemini-2.5-Pro and Gemini-3-Pro benefit from iterative planning and execution feedback, with performance increasing as the iteration budget grows. However, Gemini-3-Pro improves more rapidly, achieving substantial gains in the early iterations and reaching its peak performance earlier, around 13 iterations, whereas Gemini-2.5-Pro continues to improve until approximately 17 iterations. In addition to converging faster, Gemini-3-Pro also attains a higher peak success rate than Gemini-2.5-Pro. After reaching their respective peaks, both models exhibit saturation, indicating diminishing returns from additional iterations. These results suggest that stronger backbone models not only achieve higher final performance but also exploit execution feedback more efficiently. We also provide analysis of max detection iteration in Appendix C.

5. Analysis

We analyze CO-REDTEAM beyond benchmark performance through three lenses: evolution effect through memory, reliability of vulnerability discovery, and latency.

Memory analysis. Human security experts improve over time by accumulating and refining experience; to evaluate whether our agent exhibits similar learning dynamics, we study the effect of long-term memory initialization and evolution. We evaluate memory-driven performance evolution on CyberGym using Gemini-2.5-Pro as the backbone model. Four configurations are considered: **No memory**, where the agent operates without any long-term memory; **Static memory**, where the agent is initialized with curated security memory but memory updates are disabled; **Cold Start (Evolving)**, where the agent begins with empty memory and continuously writes new experiences; and **Warm Start (Evolving)**, where curated memory is provided as a warm start. Tasks are processed sequentially to allow memory accumulation, with performance reported as the moving average success rate (window size 100) to reveal long-horizon trends, as shown in Figure 3.

We reveal two complementary effects of long-term memory: initialization and evolution. First,

warm-started configurations outperform their cold-start counterparts in the early stages, demonstrating that curated prior knowledge provides an immediate advantage by guiding exploration and reducing unproductive actions. In particular, Static Memory improves early success rates compared to No Memory, indicating that even fixed security knowledge can bootstrap exploitation performance. Second, **memory evolution** plays a critical role in sustained improvement. Both evolving configurations exhibit an upward trend over time, while static or memory-free settings plateau early. Notably, Cold Start (Evolving) gradually closes the gap with warm-start variants, showing that the agent can autonomously acquire effective strategies from experience. The strongest performance is achieved by Warm Start (Evolving), which combines rapid early gains with continued long-term improvement, highlighting the complementary benefits of prior knowledge and continual learning. Together, these results demonstrate that long-term memory is not only useful for initialization but essential for enabling cumulative, experience-driven improvement in security exploitation tasks.

Table 3 | Recall and Precision of Detection task on BountyBench, with model Gemini-2.5-pro.

	Precision	Recall
Vanilla	0	0
OpenHands	0	0
C-agent	0.024	0.025
Co-RedTeam	0.143	0.125

Analysis on Vulnerability Discovery stage. In Table 1 and 5, we focus on the success rate of detecting vulnerabilities provided by the BountyBench. However, we notice that CO-REDTEAM usually provides zero to two vulnerabilities. Therefore, we further investigate the reliability of CO-REDTEAM in finding vulnerabilities. As shown in Table 3, we report both precision and recall of detection tasks in BountyBench with model Gemini-2.5-pro. It is obvious that CO-REDTEAM achieves significantly better results for both metrics than all baselines. Specifically, the precision of 14.3%, roughly 5x higher than the C-agent, shows that CO-REDTEAM can discover vulnerabilities much more reliably.

Table 4 | **Latency analysis:** average running time in seconds.

Agent	Model	Cybench	BountyBench	CyberGym
Vanilla	Gemini-2.5-pro	50.1	36.2	42.6
	Gemini-3-pro	43.7	34.9	37.8
OpenHands	Gemini-2.5-pro	392.1	227.5	633.5
	Gemini-3-pro	347.6	219.6	609.7
C-agent	Gemini-2.5-pro	387.2	215.3	636.4
	Gemini-3-pro	320.3	201.9	611.7
Co-RedTeam	Gemini-2.5-pro	361.5	205.4	619.7
	Gemini-3-pro	319.8	198.7	605.2

Latency analysis. Despite the performance, we also provide latency analysis to illustrate the efficiency of CO-REDTEAM. Based on Table 4, the latency analysis demonstrates that CO-REDTEAM is surprisingly efficient despite its multi-turn conversational architecture. CO-REDTEAM consistently outperforms complex agents, registering lower runtimes than both OpenHands and C-agent across all three benchmarks (e.g., 198.7s vs 219.6s for OpenHands on BountyBench). Furthermore, the

transition to Gemini-3-pro yields a universal speed improvement over Gemini-2.5-pro, reducing latency by approximately 10-15% across the board, which helps mitigate the computational cost of the more advanced iterative detection strategies.

6. Conclusion

We introduce CO-REDTEAM, a security-aware multi-agent framework for automatic vulnerability discovery and exploitation. By integrating security-domain knowledge, code-aware analysis, execution-grounded iterative reasoning, and long-term memory, CO-REDTEAM aligns agent behavior with real-world cybersecurity workflows. The effectiveness of CO-REDTEAM is validated by an over 60% success rate in exploitation tasks and over 10% absolute improvement in detection tasks.

References

- J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- D. Bhamare, M. Zolanvari, A. Erbad, R. Jain, K. Khan, and N. Meskin. Cybersecurity for industrial control systems: A survey. *computers & security*, 89:101677, 2020.
- G. Comanici, E. Bieber, M. Schaekermann, I. Pasupat, N. Sachdeva, I. Dhillon, M. Blistein, O. Ram, D. Zhang, E. Rosen, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*, 2025.
- D. Dasgupta, Z. Akhtar, and S. Sen. Machine learning in cybersecurity: a comprehensive survey. *The Journal of Defense Modeling and Simulation*, 19(1):57–106, 2022.
- Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray, and Y. Chen. Vulnerability detection with code language models: How far are we? *arXiv preprint arXiv:2403.18624*, 2024.
- R. Fang, R. Bindu, A. Gupta, Q. Zhan, and D. Kang. Llm agents can autonomously hack websites. *arXiv preprint arXiv:2402.06664*, 2024.
- M. A. Ferrag, N. Tihanyi, and M. Debbah. From llm reasoning to autonomous ai agents: A comprehensive review. *arXiv preprint arXiv:2504.19678*, 2025.
- Google. Oss-fuzz: Continuous fuzzing for open source software. <https://google.github.io/oss-fuzz/>, 2016. Large-scale open-source fuzzing infrastructure.
- J. Gottweis, W.-H. Weng, A. Daryin, T. Tu, A. Palepu, P. Sirkovic, A. Myaskovsky, F. Weissenberger, K. Rong, R. Tanno, et al. Towards an ai co-scientist. *arXiv preprint arXiv:2502.18864*, 2025.
- J. Guo, C. Wang, X. Xu, Z. Su, and X. Zhang. Repoaudit: An autonomous llm-agent for repository-level code auditing. *arXiv preprint arXiv:2501.18160*, 2025.
- P. He, Z. Li, Y. Xing, Y. Li, J. Tang, and B. Ding. Make llms better zero-shot reasoners: Structure-orientated autonomous reasoning. *arXiv preprint arXiv:2410.19000*, 2024.
- X. Huang, W. Liu, X. Chen, X. Wang, H. Wang, D. Lian, Y. Wang, R. Tang, and E. Chen. Understanding the planning of llm agents: A survey. *arXiv preprint arXiv:2402.02716*, 2024.

- J. Jang-Jaccard and S. Nepal. A survey of emerging threats in cybersecurity. *Journal of computer and system sciences*, 80(5):973–993, 2014.
- Z. Liu, Y. Zhang, P. Li, Y. Liu, and D. Yang. Dynamic llm-agent network: An llm-agent collaboration framework with agent team optimization. *arXiv preprint arXiv:2310.02170*, 2023.
- MITRE Corporation. Common Weakness Enumeration (CWE). <https://cwe.mitre.org/>, 2025. URL <https://cwe.mitre.org/>. Accessed: 2025-01-XX.
- Y. Nong, M. Aldeen, L. Cheng, H. Hu, F. Chen, and H. Cai. Chain-of-thought prompting of large language models for discovering and fixing software vulnerabilities. *arXiv preprint arXiv:2402.17230*, 2024.
- S. Ouyang, J. Yan, I. Hsu, Y. Chen, K. Jiang, Z. Wang, R. Han, L. T. Le, S. Daruki, X. Tang, et al. Reasoningbank: Scaling agent self-evolving with reasoning memory. *arXiv preprint arXiv:2509.25140*, 2025.
- OWASP Foundation. OWASP Top 10: The Ten Most Critical Web Application Security Risks. <https://owasp.org/Top10/>, 2021. URL <https://owasp.org/Top10/>. Accessed: 2025-01-XX.
- I. H. Sarker, A. Kayes, S. Badsha, H. Alqahtani, P. Watters, and A. Ng. Cybersecurity data science: an overview from machine learning perspective. *Journal of Big data*, 7(1):41, 2020.
- A. Singhanian, C. Dupuy, S. Mangale, and A. Namboori. Multi-lingual multi-turn automated red teaming for llms. *arXiv preprint arXiv:2504.03174*, 2025. doi: 10.48550/arXiv.2504.03174.
- B. Steenhoek, M. M. Rahman, M. K. Roy, M. S. Alam, H. Tong, S. Das, E. T. Barr, and W. Le. To err is machine: Vulnerability detection challenges llm reasoning. *arXiv preprint arXiv:2403.17218*, 2024.
- N. Sun, J. Zhang, P. Rimba, S. Gao, L. Y. Zhang, and Y. Xiang. Data-driven cybersecurity incident prediction: A survey. *IEEE communications surveys & tutorials*, 21(2):1744–1772, 2018.
- O. Team. Openhands: An open platform for general-purpose ai agents. <https://github.com/All-Hands-AI/OpenHands>, 2024. Accessed: 2025-XX-XX.
- S. Ullah, M. Han, S. Pujar, H. Pearce, A. Coskun, and G. Stringhini. Llms cannot reliably identify and reason about security vulnerabilities (yet?): A comprehensive evaluation, framework, and benchmarks. In *2024 IEEE symposium on security and privacy (SP)*, pages 862–880. IEEE, 2024.
- Z. Wang, T. Shi, J. He, M. Cai, J. Zhang, and D. Song. Cybergym: Evaluating ai agents’ cybersecurity capabilities with real-world vulnerabilities at scale. *arXiv preprint arXiv:2506.02548*, 2025.
- J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- R. Widyasari, M. Weyssow, I. C. Irsan, H. W. Ang, F. Liauw, E. L. Ouh, L. K. Shar, H. J. Kang, and D. Lo. Let the trial begin: A mock-court approach to vulnerability detection using llm-based agents. *arXiv preprint arXiv:2505.10961*, 2025.
- D. Xu, I. Gondal, X. Yi, T. Susnjak, P. Watters, and T. R. McIntosh. The erosion of cybersecurity zero-trust principles through generative ai: A survey on the challenges and future directions. *Journal of Cybersecurity and Privacy*, 5(4):87, 2025.

- A. K. Zhang, N. Perry, R. Dulepet, J. Ji, C. Menders, J. W. Lin, E. Jones, G. Hussein, S. Liu, D. Jasper, et al. Cybench: A framework for evaluating cybersecurity capabilities and risks of language models. *arXiv preprint arXiv:2408.08926*, 2024.
- A. K. Zhang, J. Ji, C. Menders, R. Dulepet, T. Qin, R. Y. Wang, J. Wu, K. Liao, J. Li, J. Hu, et al. Bountybench: Dollar impact of ai agent attackers and defenders on real-world cybersecurity systems. *arXiv preprint arXiv:2505.15216*, 2025a.
- J. Zhang, H. Bu, H. Wen, Y. Liu, H. Fei, R. Xi, L. Li, Y. Yang, H. Zhu, and D. Meng. When llms meet cybersecurity: A systematic literature review. *Cybersecurity*, 8(1):55, 2025b.
- X. Zhou, T. Zhang, and D. Lo. Large language model for vulnerability detection: Emerging results and future directions. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*, pages 47–51, 2024.
- T. Y. Zhuo, D. Wang, H. Ding, V. Kumar, and Z. Wang. Training language model agents to find vulnerabilities with ctf-dojo. *arXiv preprint arXiv:2508.18370*, 2025.